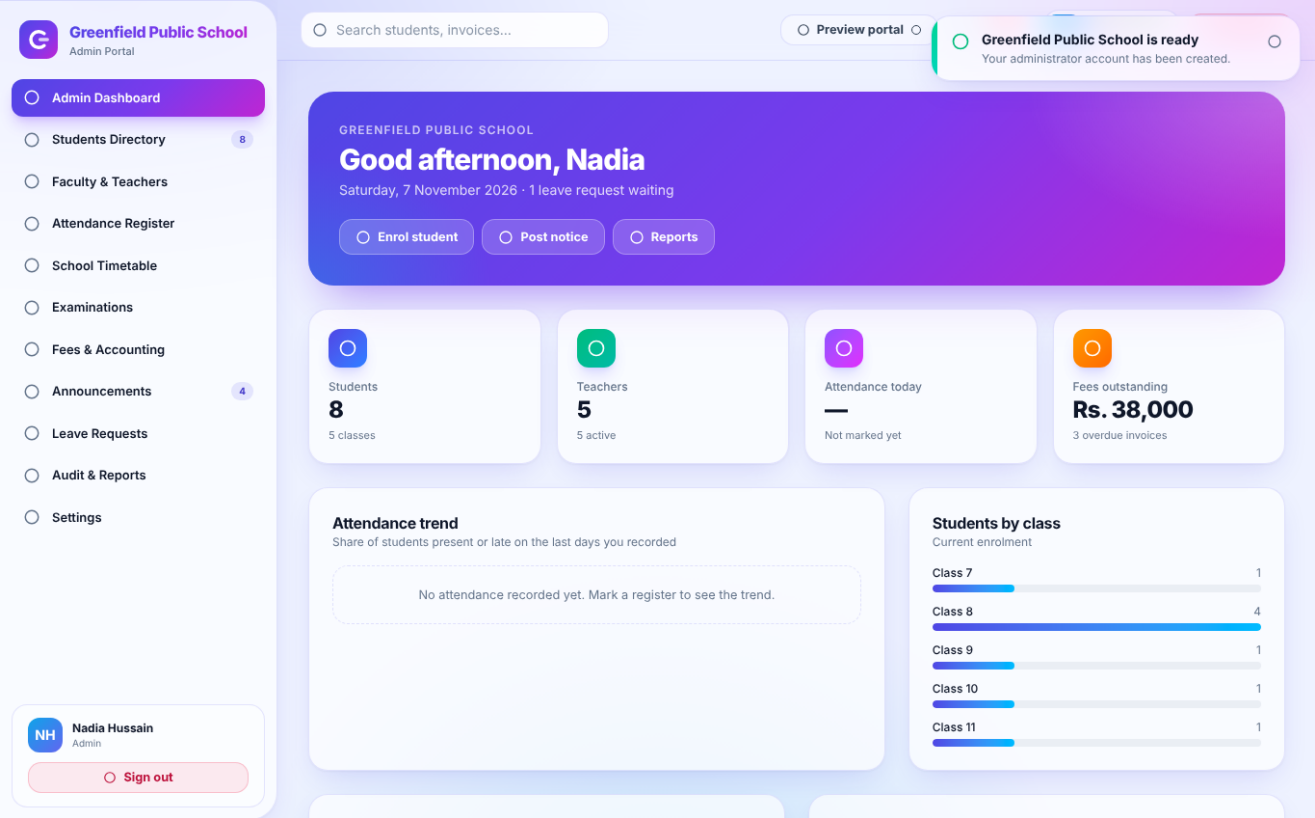
CampusFlow

A role-based school management web app · React 19, TypeScript, Vite, Tailwind CSS v4

Zain ul Haseeb · Web developer



Overview

CampusFlow brings the everyday work of a school into one application: admissions, attendance, timetables, exams, fee collection, accounting and parent communication. Five separate portals (admin, teacher, accountant, parent and student) each show only what that role needs. The app runs fully in the browser, so it can be tried instantly with no server setup.

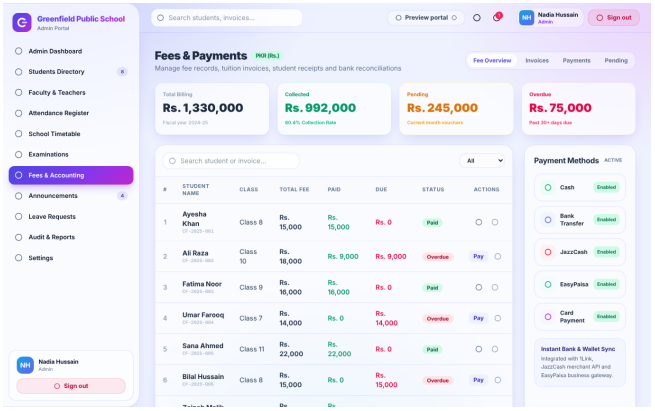
The problem

Schools often juggle paper registers, spreadsheets and chat groups. Fee records drift from attendance records, and parents have no clear view of their child. I wanted one consistent system where every number on screen comes from a real record, not typed-in demo values.

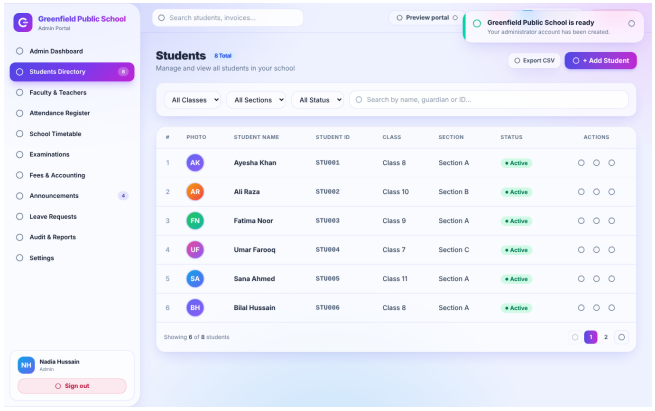
Key features

- **Role-based portals** for admin, teacher, accountant, parent and student, with permission checks on every action.
- **Student directory** with admissions, profiles and printable ID cards.
- **Attendance register** feeding live percentages, class ranks and dashboards.
- **Fees & accounting:** invoices that turn overdue automatically, validated payments, receipts, a payments ledger, expenses, budgets and payroll.
- **Exams** with mark entry and report cards; leave requests; announcements; parent–teacher meeting booking.
- **Light and dark themes**, responsive layouts from phone to desktop, loading skeletons and keyboard/screen-reader support.

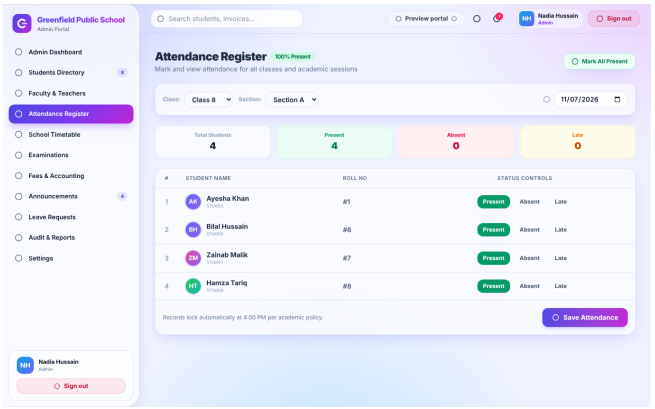
Screens



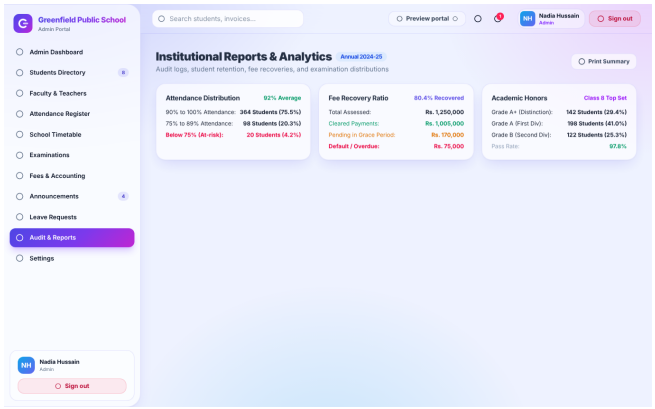
Fees & accounting with live totals



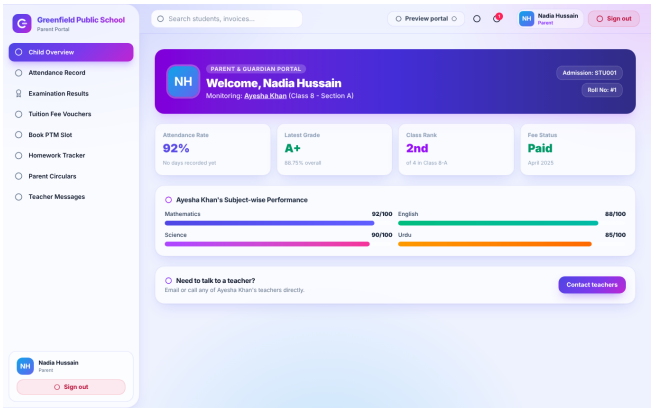
Student directory



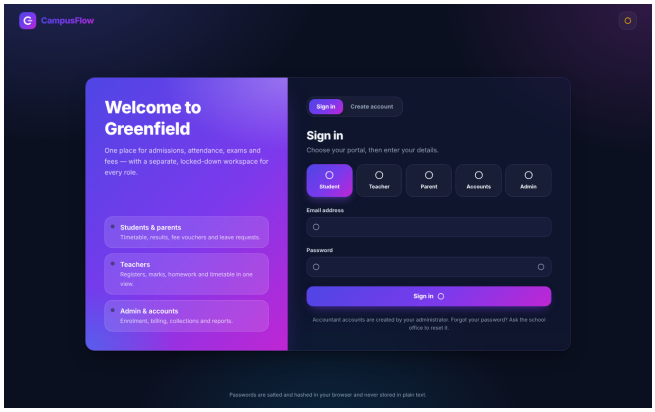
Attendance register



Audit & reports



Parent portal



Sign-in screen in dark mode

Security

- Passwords hashed with PBKDF2-SHA256 (210,000 iterations) using the Web Crypto API; no plain-text passwords stored.
- Login lockout after repeated failures; sessions end after 30 minutes idle or 8 hours total; signing out in one tab signs out all tabs.
- First-run setup creates the admin; teachers need an admin-issued staff code; parents must match an existing admission number.
- All input is cleaned and validated; payment amounts and image uploads are checked before saving.
- Content-Security-Policy and HSTS headers configured for deployment.

Design

An indigo → violet → fuchsia gradient with glassmorphism cards over a soft aurora background. Sora is used for headings and Plus Jakarta Sans for body text. Reduced-motion and print styles are included.

Performance & SEO

- Lazy-loaded screens and split bundles keep the first load small; shimmer skeletons cover loading states.
- Meta, Open Graph, Twitter and JSON-LD tags, web manifest, robots and sitemap generation, and an accessible skip link.

Tech stack

Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, lucide-react
Data layer	Browser localStorage with validated, role-guarded actions
Cloud-ready	Supabase schema with row-level security included (not yet wired to the screens)
Testing	Scripted browser tests with Playwright for sign-up, roles, lockout and payments

Next steps

- Connect the screens to the Supabase backend so data is shared across devices.
- Add real payment-gateway and SMS/email notification integrations.